
Gen AI Hands On - 2

Name: Suchitra Shankar

Class: 6J

SRN: PES2UG23CS608

Date: 26 Feb 2026

Email: suchitrashankar07@gmail.com

Phone number: 6366250274

Github link: <https://github.com/SuchitraShankar07/GenAI2026>

Overview:

This submission consists of five major parts covering foundational and advanced concepts in Generative AI using LangChain, Prompt Engineering, Retrieval-Augmented Generation (RAG), and Mixture of Experts (MoE).

The objective of this unit was to understand how Large Language Models (LLMs) function, how to control their behavior using structured prompts, and how to build scalable AI systems using vector databases and modular architectures.

Part 1: LangChain Foundations

Key Concepts Covered

1. Why LangChain

LangChain provides a unified interface for interacting with multiple LLM providers. It abstracts API-specific implementations, allowing easy switching between models with minimal code changes.

2. Tokens and Context Windows

LLMs operate on tokens rather than words.

Cost and memory usage are directly tied to token count.

Each model has a context window limit, beyond which earlier tokens are truncated.

3. Temperature Parameter

Temperature controls randomness in output generation:

0.0 → deterministic and consistent responses

1.0 → creative and varied responses

Experiments demonstrated that lower temperatures produce consistent outputs, while higher temperatures increase variability.

4. Prompts and Parsers

System, Human, and AI message roles control behavior.

ChatPromptTemplate ensures safe and structured formatting.
StrOutputParser extracts clean text from model responses.

5. LCEL (LangChain Expression Language)

LCEL enables composable pipelines using the pipe (|) operator.

Pipeline structure:

Input → Template → Model → Parser → Output

This improves modularity, scalability, and maintainability.

Part 2: Prompt Engineering

2a: Anatomy of a Prompt

CO-STAR Framework

Prompts were structured using:

- Context
- Objective
- Style
- Tone
- Audience
- Response Format

Structured prompts produced significantly more controlled and predictable outputs compared to generic prompts.

Hallucination vs Creativity

The model fills missing information probabilistically. Ambiguous prompts lead to fabricated details. Explicit constraints reduce hallucination.

2b: Zero-Shot vs Few-Shot Learning

Zero-Shot :The model relies purely on pre-trained knowledge.

Few-Shot: Providing examples in the prompt enables in-context learning.

The model detects patterns from examples and replicates structure and tone.

Key Insight:

Example quality directly impacts output quality.

2c: Hard Prompts vs Fine-Tuning

Method	Description	Cost	Use Case
Prompt Engineering	Modify input text	Low	General tasks
Fine-Tuning	Modify model weights	High	Domain-specific tasks

Prompt engineering is faster and cost-effective for most applications.

Dynamic few-shot selection using templates improves scalability.

Part 3: Advanced Reasoning Techniques

3a: Chain of Thought (CoT)

CoT forces the model to generate intermediate reasoning steps before producing a final answer.

Example:
Standard Prompt → Direct answer
CoT Prompt → Step-by-step reasoning → Correct output

Observation:
Smaller models significantly benefit from explicit reasoning instructions.

3b: Tree of Thoughts (ToT)

ToT explores multiple solution branches before selecting the best one.

Process:

1. Generate multiple candidate solutions.
2. Use a judge prompt to evaluate and select the optimal one.

Use Case:

- Strategic decision-making
- Brainstorming

Higher computational cost but improved robustness.

3c: Graph of Thoughts (GoT)

GoT allows divergence and convergence:

- Generate multiple independent drafts.
- Aggregate insights.
- Refine into a final solution.

Use Case:

- Creative writing
- Research synthesis
- Multi-perspective reasoning

This approach increases output quality at the cost of latency.

Part 4: RAG and Vector Stores

4a: Embeddings

Concept

Embeddings convert text into numerical vectors where semantically similar words are closer in vector space.

Example:

- Cat vs Dog → Higher cosine similarity
- Cat vs Car → Lower cosine similarity

Cosine similarity is the mathematical foundation of semantic search.

4b: Naïve RAG Pipeline

RAG consists of three steps:

1. Retrieval – Convert query into vector and search database
2. Augmentation – Inject retrieved context into prompt
3. Generation – Produce grounded response

FAISS was used as the vector store.

Outcome:

The model successfully answered questions using only retrieved external context.

4c: Indexing Algorithms

At large scale, brute-force search becomes inefficient. FAISS supports optimized indexing methods.

Flat Index

- 100% accurate
- Slow
- Suitable for small datasets

IVF (Inverted File Index)

- Uses clustering
- Faster search
- Moderate accuracy trade-off

HNSW

- Graph-based search
- Very fast
- Higher memory usage

Product Quantization (PQ)

- Compresses vectors
- Reduces memory
- Slight accuracy loss

Key Insight:

There is always a trade-off between speed, memory, and accuracy.

Part 5: Assignment – Mixture of Experts (MoE)

Objective

To build a Smart Customer Support Router using Groq API.

Architecture

The system consists of:

1. Router – Classifies user query
2. Expert Modules – Specialized system prompts
3. Orchestrator – Routes input to correct expert

Experts Implemented

- Technical Expert – Handles debugging and code issues
- Billing Expert – Handles refunds and subscription queries
- Sales Expert – Handles pricing and product inquiries
- General Expert – Fallback for general queries

Each expert uses the same base model but different system prompts.

Routing Logic

The router:

- Uses temperature = 0
- Classifies intent into: technical, billing, sales, general
- Returns only the category name

If classification fails, it defaults to "general".

Orchestration Flow

1. User query is sent to router.
2. Router returns category.
3. Corresponding system prompt is selected.
4. Expert model generates final response.

This modular design improves scalability and specialization.
